# Python Notes — Iteration Behind the Scenes

*Topics: Iterables | Iterators | Iteration Protocol | File Reading | Interview POV*

## 1. The Big Picture — What Actually Happens in a for Loop?

Every time you write a for loop, Python is quietly running a two-step process under the hood that most beginners never see. Understanding this process is one of the most commonly tested concepts in Python interviews.

> 💡 **Tip:** When an interviewer asks 'how does a for loop work internally?' — this section is the answer.

Python's iteration model is built on two key concepts: Iterables and Iterators.

| Term | What it is | Example |
|---|---|---|
| `Iterable` | Any object you can loop over. Has __iter__() method. | list, string, file, set, dict |
| `Iterator` | An object that tracks position and delivers next item. Has __next__() method. | iter(myList), open('file') |
| `StopIteration` | Exception raised when no more items remain. | Signals end of iteration |

When you write 'for item in myList', Python secretly does this:

```
# What Python does behind the scenes:
iterator = iter(myList)          # Step 1: calls myList.__iter__()
while True:
    try:
        item = next(iterator)    # Step 2: calls iterator.__next__()
        # ... run the loop body ...
    except StopIteration:
        break                    # Step 3: loop ends cleanly
```

> **Q: What does a for loop actually do internally?**
> A: It calls iter() on the object to get an iterator, then repeatedly calls next() on that iterator until a StopIteration exception is raised, at which point the loop exits cleanly.

## 2. Iterables vs Iterators — The Crucial Difference

This is one of the most common Python interview questions. Many candidates confuse the two. The difference is clear once you see it in practice.

## Iterable — the container

An iterable is any object that knows how to produce an iterator. It has a __iter__() method. You can loop over it, but it does NOT track position itself.

```
myList = [1, 2, 3, 4]

iter(myList) is myList    # False — a NEW iterator object is created
each time
iter(myList) is myList    # False — calling iter() again makes another
fresh one
```

📝 **Note:** Because iter(myList) creates a new object every time, a list can be looped over again and again from the beginning. Each loop gets a fresh iterator.

## Iterator — the cursor

An iterator is the object that actually tracks where you are in the sequence. It has both __iter__() and __next__(). Once exhausted, it's done — it cannot reset.

```
myList = [1, 2, 3, 4]
I = iter(myList)          # creates an iterator

I.__next__()              # 1
I.__next__()              # 2
I.__next__()              # 3
I.__next__()              # 4
I.__next__()              # StopIteration — no more items!
```

## File objects — a special case

```
f = open('chai.py')

iter(f) is f              # True  — file IS its own iterator
iter(f) is f.__iter__()   # True  — same object returned each time
```

A file object is both an iterable AND its own iterator. Calling iter() on it returns itself — not a new object. This means a file can only be read through once without reopening it.

**Q: What is the difference between an iterable and an iterator?**

A: An iterable is any object with __iter__() — it can produce an iterator (e.g. list, string, file). An iterator is an object with __next__() that tracks position and delivers items one at a time. All iterators are iterables, but not all iterables are iterators. A list is iterable but not an iterator; iter(list) gives you an iterator.

**Q: Why does iter(myList) is myList return False, but iter(f) is f return True for a file?**

A: A list is a re-usable iterable — each call to iter() creates a brand new iterator object so the list can be looped from scratch multiple times. A file object is its own iterator and always returns itself from iter(), meaning it maintains a single read position and cannot be rewound without reopening.

## 3. The Iteration Protocol

Python's iteration system is built on a formal protocol — a set of rules any object can follow to become iterable or an iterator.

| Method | Required by | What it does |
|---|---|---|
| `__iter__()` | Iterable | Returns an iterator object |
| `__next__()` | Iterator | Returns the next item, or raises StopIteration |
| `__iter__() on iterator` | Iterator | Returns self (so iterators can be used in for loops directly) |

Any object that implements both __iter__() and __next__() is a full iterator. Any object that implements only __iter__() (returning some other iterator) is just an iterable.

💡 **Tip:** This protocol is why you can use any custom class in a for loop — just implement __iter__() and __next__() and Python will treat it like a list.

## 4. Reading Files — Three Methods Compared

Your REPL session showed three distinct ways to read a file. Interviewers often ask which approach is best and why.

### Method 1 — readline() (line by line, manual)

```
f = open('chai.py')
f.readline()    # reads one line, moves cursor forward
f.readline()    # reads the next line
# ...
f.readline()    # returns '' (empty string) when file is finished
```

Each call to `readline()` moves the internal file cursor forward by one line. When the end of the file is reached, it returns `''` (an empty string — not None). This is how you detect EOF manually.

## Method 2 — for loop over file (recommended)

```
for line in open('chai.py'):
    print(line)
```

This is the most Pythonic approach. The file object is an iterator, so Python calls __next__() on it for each iteration, reading one line at a time. It is memory-efficient because the entire file is never loaded at once.

## Method 3 — while loop with readline()

```
f = open('chai.py')
while True:
    line = f.readline()
    if not line: break      # empty string is falsy
    print(line, end='')
```

This uses the fact that an empty string `''` is falsy in Python. When `readline()` returns `''`, the `if not line` condition is True, and `break` exits the loop.

> ⚠️ **Warning:** An empty line in the file returns '\n' (a newline character), which is truthy. Only the end of file returns '' (truly empty). Don't confuse them.

| Method | Memory use | Best for | Notes |
|---|---|---|---|
| `readline()` | Low | Manual/partial reads | Cursor moves forward; '' means EOF |
| `for line in file` | Low | Most cases — Pythonic | Cleanest syntax; recommended |
| `while + readline()` | Low | Custom control flow | Useful when you need more logic mid-read |
| `readlines()` | High | Small files only | Loads ALL lines into a list at once |
| `read()` | High | Small files only | Loads entire file as one string |

**Q: What is the most memory-efficient way to read a large file in Python?**

A: Iterate over the file object directly with 'for line in f' or use readline() in a while loop. Both read one line at a time without loading the entire file into memory. Avoid read() or readlines() for large files as they load everything at once.

## 5. Truthiness and Falsiness in Python

Your session demonstrated this with the 'if not test' examples. Python treats certain values as False even though they are not the boolean False. This is called falsiness.

```
test = "shwetank"
if not test:              # non-empty string is truthy → block does NOT
run
    print("chai")

test = ""
if not test:              # empty string is falsy → block RUNS
    print("chai")        # Output: chai
```

### Values that are Falsy in Python

| Falsy Value | Type | Truthy Equivalent |
|---|---|---|
| `False` | bool | True |
| `0` | int / float | Any non-zero number |
| `"" or ''` | str | Any non-empty string |
| `[]` | list | Non-empty list |
| `{}` | dict / set | Non-empty dict or set |
| `()` | tuple | Non-empty tuple |
| `None` | NoneType | Any actual value |
| `b""` | bytes | Non-empty bytes |

This is why `if not line: break` works as an EOF check — an empty string `''` is falsy, so `not ''` is `True`.

**Q: Why does 'if not line' work to detect end-of-file when using readline()?**
A: When readline() reaches the end of a file, it returns an empty string ''. An empty string is falsy in Python, so 'not line' evaluates to True, triggering the break. An empty line within the file returns '\n', which is truthy and does not trigger the break.

## 6. StopIteration — How Loops End Cleanly

When an iterator runs out of items, Python raises a StopIteration exception. This is not an error — it is the official signal that iteration is complete. The for loop catches it automatically and exits.

```
I = iter([1, 2, 3])
next(I)     # 1
next(I)     # 2
next(I)     # 3
next(I)     # raises StopIteration

# In a for loop, Python catches StopIteration silently and stops the
loop
```

```
# Same with a file:
f = open('chai.py')
f.__next__()     # first line
# ...
f.__next__()     # after last line → StopIteration
```

**Q: What is StopIteration and when is it raised?**

A: StopIteration is a built-in exception raised by an iterator's __next__() method when there are no more items to return. It is not an error — it is the formal signal that iteration is complete. Python's for loop catches it automatically and exits the loop. You only see it directly if you call __next__() or next() manually.

## 7. Building a Custom Iterator (Interview Favourite)

Interviewers often ask you to write a class that implements the iteration protocol. Here is the pattern to know:

```
class CountUp:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):       # makes it an iterable
        return self           # returns itself — it is also its own
iterator

    def __next__(self):       # makes it an iterator
        if self.current >= self.limit:
            raise StopIteration
        self.current += 1
        return self.current

for num in CountUp(5):
    print(num)   # 1  2  3  4  5
```

💡 **Tip:** Notice that __iter__ returns self. This is the same pattern a file uses — it is both iterable and its own iterator in one object.

**Q: How do you make a custom class work in a for loop?**

A: Implement two dunder methods: __iter__() which returns an iterator object (often self), and __next__() which returns the next value or raises StopIteration when exhausted. This is called the iterator protocol and is the foundation of all iteration in Python.

## 8. Generators — Iterators the Easy Way

A generator is a simpler way to create an iterator without writing a full class. Any function with a yield statement returns a generator object, which is automatically an iterator.

```
def count_up(limit):
    current = 1
    while current <= limit:
        yield current        # pauses here and returns the value
        current += 1         # resumes from here on next call

for num in count_up(5):
    print(num)  # 1  2  3  4  5
```

Each time `yield` is hit, the function pauses and returns the value. The next call to `__next__()` resumes execution right after the `yield`. When the function ends, `StopIteration` is raised automatically.

| | Custom Iterator Class | Generator Function |
|---|---|---|
| `How created` | Class with __iter__ + __next__ | Function with yield |
| `State tracking` | Manual (self.current etc.) | Automatic (local variables preserved) |
| `Verbosity` | More code | Much shorter |
| `Memory` | One item at a time | One item at a time |
| `Use when` | Need full class with other methods | Simple sequence generation |

**Q: What is a generator and how does it differ from a regular function?**

A: A generator is a function that uses yield instead of return. Calling it returns a generator object (an iterator) rather than executing the body immediately. Each call to next() on the generator runs the function until the next yield, pauses, and returns that value. Local variable state is preserved between calls. When the function body ends, StopIteration is raised automatically.

## 9. Quick Reference — Interview Cheat Sheet

| Concept | One-line answer |
|---|---|
| `What is an iterable?` | Any object with __iter__(); can be looped over |
| `What is an iterator?` | Object with __next__(); tracks position, delivers items one at a time |
| `How does for work internally?` | iter() → repeated next() → catches StopIteration → exits |
| `iter(list) is list?` | False — a NEW iterator object is created |
| `iter(file) is file?` | True — file is its own iterator |
| `How to detect EOF with readline()?` | readline() returns '' (empty string, falsy) at end of file |
| `What raises StopIteration?` | __next__() when no more items remain |
| `What is a generator?` | A function with yield; returns a lazy iterator automatically |
| `Falsy values?` | False, 0, '', [], {}, (), None |
| `Most memory-efficient file read?` | for line in f: or readline() in a while loop |

## 10. Common Interview Errors & Traps

⚠️ **Trap 1:** Calling next() on an exhausted iterator raises StopIteration — it does NOT reset. To re-iterate, you must call iter() again on the original iterable (not on the old iterator).

⚠️ **Trap 2:** A file object exhausted by one for loop cannot be re-read by another loop in the same session — you must reopen the file with open() because iter(f) is f (same exhausted object).

⚠️ **Trap 3:** Empty line in a file returns '\n' (truthy), NOT '' (falsy). Only end-of-file returns ''. Many candidates get this wrong in readline() while-loop patterns.

⚠️ **Trap 4:** Not all iterables are iterators. A list is iterable but has no __next__(). Calling next() directly on a list raises TypeError.

⚠️ **Trap 5:** A generator can only be consumed once. After StopIteration is raised, it is exhausted forever — unlike a list which can be iterated over and over.